

— Project information —

Please read the instructions below carefully. They will answer most of the questions you might have about the project.

Schedule

- **From now on:** Start forming groups and choose topics from the list we provide (see below).
- **June 25:** During lecture: Opportunity for open questions about topics, groups, etc.
- **June 28 (midnight):** Deadline for final formation of groups and project registration via google form (<https://forms.gle/2NicBRv8ADUwTKW1A>). In case you face difficulty, register via email to Aayush (aayush.mishra@tu-dortmund.de). For more information refer to the **Guidelines** and **Project** sections.
- **In July:** During lecture and exercise sessions: Opportunity to ask questions and discuss your project with the teaching staff. Outside of lecture sessions, use Moodle discussion forum to ask general questions, or contact lecturers via e-mail (note that we may not be able to respond to every question that way).
- **July 19 (midnight):** Deadline for submission of the presentation slides via google form (<https://forms.gle/sw5immNjiQdvxQrV7>). In case you face difficulty, you can submit via email to Aayush (aayush.mishra@tu-dortmund.de). For more details, refer to the **Presentation** section.
- **July 20 - 24:** Presentations, split into multiple sessions of about 5-6 presentations, depending on the number of groups. Exact dates and location will be announced. You are only allowed to attend the session in which you present. Students from other groups not presenting in the session will not be allowed to attend.
- **August 23 (midnight):** Deadline for report submission via google form (<https://forms.gle/zPUXu3Uu6JQMza6Z6>). In case you face difficulty, you can submit via email to Aayush (aayush.mishra@tu-dortmund.de). For more details, refer to the **Report** section.

Guidelines

Groups The project is meant to be done in groups of 2. It is possible to do it in groups of 1 or 3, however the scope for a 3 person group will be increased accordingly, while the scope for a 1 person group will be the same as for 2. While we allow groups of different sizes to accommodate your needs, we strongly encourage groups of 2.

Previous Courses Without previous experience in data analysis projects and reports, this project work may be very hard. Thus, having previously taken courses such as Case Studies is highly recommended (but not mandatory).

Registration Please form your groups and register via google form (<https://forms.gle/2NicBRv8ADUwTKW1A>) before the deadline (**June 28, 23:59**). The deadline is strict and any registrations after the deadline will not be considered. In case you face difficulty with valid reason, register via email to Aayush (aayush.mishra@tu-dortmund.de) containing the following information:

- Names, matriculation numbers, study programmes, and module (applied or methods; if applicable) of all group members.
- Preferred topic(s) for your project (see below).
- If you find your own project: The name of the teaching staff who has approved your project (see below).

We will announce the group numbers and allocated topics on Moodle.

Topics

To find a topic, you can do one of the two following options

- Select *up to three topics* of your preference from the list provided by us below. We will do our best to assign you a topic that you prefer, but if there is too much overlap with other groups, we may assign you to a project you did not select to avoid excessive repetition of the topics.
- Come up with your own topic. The topic needs to be selected such that using BayesFlow for ABI is reasonable. The topic must be discussed and **approved** by the teaching staff beforehand. To initiate the approval process, please talk to any member of the teaching staff. Note that if you chose to discuss this by email, it may take a couple of days to get your idea approved (depending on the amount of discussion needed).

Topics Here is the list of topics that you can choose from (unless you come up with a topic of your own). You can consult each project with its respective lecturer indicated below.

1. Double Responses in Decision Tasks

Lecturer: Simon (simon.kucharsky@tu-dortmund.de)

In many cognitive experiments, participants are asked to make quick decisions under time pressure, such as identifying a stimulus, choosing between alternatives, or categorizing an input. These are known as *speeded decision tasks*, and typically provide data in a form of choices and response times. To analyze such data, researchers often turn to *evidence accumulation models* (EAMs), which offer a principled statistical framework for modeling how decisions unfold over time.

A persistent phenomenon observed during speeded decision tasks is *double responding*. For example, in a lexical decision task where participants judge whether a letter string is a word or non-word, a participant might first press a key indicating “word” and then quickly follow it with another press indicating “non-word.” These double responses are often excluded during data processing or even not recorded at all. However, they may actually reflect moments of uncertainty or conflict in the decision process and could contain informative statistical signals.

In this project, you will explore how incorporating double responses into an EAM affects inference and model fit. Choose a model from Evans et al. (2020) that extends an EAM (such as the RDM) to account for double responses. Implement the model using **BayesFlow**. Estimate the parameters on data from Evans et al. (2020).

Evans, N. J., Dutilh, G., Wagenmakers, E.-J., & van der Maas, H. L. (2020). Double responding: A new constraint for models of speeded decision making. *Cognitive Psychology*, 121, 101292. <https://doi.org/10.1016/j.cogpsych.2020.101292>

2. Collapsing Decision Boundaries

Lecturer: Simon (simon.kucharsky@tu-dortmund.de)

In many cognitive experiments, participants are asked to make quick decisions, such as identifying a stimulus, choosing between alternatives, or categorizing an input. These are known as *speeded decision tasks*, and they provide data in the form of both choices and response times. A key question is how people adjust their decision-making strategies when pressured to respond quickly, potentially at the cost of accuracy.

To study these processes, researchers often use evidence accumulation models (EAMs) which simulate decisions as a process of accumulating noisy evidence over time until a threshold (“decision boundary”) is reached. One central modeling challenge is how to represent the decision boundary: should it remain fixed, collapse over time, or be modulated by a rising urgency signal?

A study by Katsimpokis et al. (2020) showed that the answer depends in part on *how time pressure is implemented* in the experiment. When participants were instructed to respond quickly, fixed boundaries seemed to fit best. However, when time pressure was enforced via an explicit deadline, *linearly collapsing boundaries* provided a better account of the data.

What remains unexplored is whether other mechanisms could improve the fit even further, such as non-linearly collapsing boundaries (e.g., exponential or sigmoid functions) or urgency gating mechanisms.

In this project, you will extend previous modeling work by implementing non-linear boundary functions or urgency gating mechanisms within an EAM of your choice. Implement an EAM model with non-linear collapsing boundaries/urgency gating mechanisms in BayesFlow. Fit it to empirical response time and choice data from the Katsimpokis et al. (2020) study and discuss the results.

Katsimpokis, D., Hawkins, G. E., & van Maanen, L. (2020). Not all speed-accuracy trade-off manipulations have the same psychological effect. *Computational Brain & Behavior*, 3(3), 252-268. <https://doi.org/10.1007/s42113-020-00074-y>

3. The SWIFT Model of Eye Movements

Lecturer: Simon (simon.kucharsky@tu-dortmund.de)

In reading research, eye-tracking provides data relevant to how people process written language. One of the key questions is how readers decide when to move their eyes and where to look next. These decisions are influenced by both the cognitive state of the reader and properties of the text, such as word frequency or sentence structure. Modeling these dynamics statistically is challenging due to the complexity and variability of eye movement behavior.

One approach to modeling these processes is the *SWIFT model*, a dynamic generative model of eye movement control during reading. It simulates how a reader's gaze shifts across a sentence as they process its content. The model incorporates timing and control mechanisms for both *fixation duration* (how long the eye stays on a word) and *saccades* (eye movements to the next word). The full version of SWIFT is computationally intensive and has an intractable likelihood. However, you can work with a simplified version of the SWIFT model introduced by Engbert & Rabe (2024).

Implement the simplified SWIFT model in **BayesFlow**. Use eye-tracking data to estimate model parameters related to gaze control and reading dynamics. Investigate how well the model captures the observed fixation durations and movement patterns.

You will work with real eye-tracking data from a controlled reading experiment. One example dataset, available at <https://osf.io/tey4d>, contains fixation sequences for an individual participant. You can also link the fixation data to word properties using the corpus file from <https://osf.io/nj2mf>.

Engbert, R., & Rabe, M. M. (2024). A tutorial on Bayesian inference for dynamical modeling of eye-movement control during reading. *Journal of Mathematical Psychology*, 119, 102843. <https://doi.org/10.1016/j.jmp.2024.102843>

4. Inference of protein secondary structure motifs

Lecturer: Simon (simon.kucharsky@tu-dortmund.de)

Proteins are long chains of amino acids that fold into specific shapes. One key level of organization is the secondary structure, where each amino acid is part of three local folding patterns (alpha-helix, beta-sheet or random coil [1]), which then further fold into three-dimensional structures, defining the function of the protein. In this project, we focus specifically on predicting alpha-helix patterns using a two-state Hidden Markov Model (HMM) [2]. The two states are “alpha-helix” and “other” (encompassing beta-sheets and coils). We assume fixed emission and transition probabilities derived from empirical data [3].

We define the following generative model for simulating amino acid sequences.

The sequence always starts in the “other” secondary structure state. The sequence of states are generated using a first order Markov chain with the following transition probabilities: If the current state is “alpha-helix”, the next state is “alpha-helix” with probability $p = 90\%$ and “other” with probability $1 - p = 10\%$. If the the current state is “other”, the next state is “alpha-helix” with probability $p = 0.05$ and “other” with probability $1 - p = 95\%$.

We output amino acids based on the following tables of emission probabilities, conditionally on the secondary structure state:

alpha-helix

A	R	N	D	C	E	Q	G	H	I
12%	6%	3%	5%	1%	9%	5%	4%	2%	7%
L	K	M	F	P	S	T	W	Y	V
12%	6%	3%	4%	2%	5%	4%	1%	3%	6%

other

A	R	N	D	C	E	Q	G	H	I
6%	5%	5%	6%	2%	5%	3%	9%	3%	5%
L	K	M	F	P	S	T	W	Y	V
8%	6%	2%	4%	6%	7%	6%	1%	4%	7%

Using this simulator, we can simulate amino acid chains of arbitrary length. With the Forward-Backward algorithm [4], it is possible to infer state probabilities for a given amino acid sequence, e.g. using the `hmmlearn` Python package [5]. Given pairs of amino acid sequences and state probabilities as training data, the goal is to use BayesFlow to train a neural posterior density estimator that predicts the state probabilities given a sequence of amino acids.

Compare the posterior state probability estimates for a new protein sequence to the known ground truth, for example, the annotated secondary structure of human insulin [6].

References:

1. <https://old-ib.bioninja.com.au/higher-level/topic-7-nucleic-acids/73-translation/protein-structure.html>
2. https://scholar.harvard.edu/files/adeqirmenci/files/hmm_adeqirmenci_2014.pdf
3. <https://www.kaggle.com/datasets/alfrandom/protein-secondary-structure>
4. <https://web.stanford.edu/~jurafrsky/slp3/A.pdf>
5. <https://pypi.org/project/hmmlearn/>
6. <https://www.rcsb.org/3d-sequence/1A7F>

5. Estimating Cosmological Parameters from Matter Power Spectrum

Lecturer: Aayush (aayush.mishra@tu-dortmund.de)

The large-scale structure of the universe encodes information about its composition, expansion history, and initial conditions [4]. The matter power spectrum, $P(k)$, describes how matter density fluctuations vary with scale and is one of the central tools in modern cosmology [1, 4]. Its shape and amplitude are sensitive to the Hubble constant (H_0), the matter density (Ω_m), the spectral index (n_s), and the primordial amplitude (A_s). Accurate inference of these parameters helps constrain Λ CDM.

The goal of this project is to use `BayesFlow` to infer the posterior distribution over $\theta = \{H_0, \Omega_m, n_s, A_s\}$ given a noisy observation of the linear matter power spectrum $P(k)$ at redshift $z = 0$, where k is the wavenumber. You will build an amortized neural posterior estimator (NPE) with jointly trained summary and posterior networks, then assess its calibration on simulated test data.

Simulator. Generate $P(k)$ with `CAMB` [2] at $z = 0$ (linear spectrum) on a fixed grid of ~ 100 log-spaced wavenumbers over $k \in [10^{-3}, 1] h \text{ Mpc}^{-1}$, with $P(k)$ in $(\text{Mpc}/h)^3$. Note that `CAMB` is parameterized by *physical* densities, not Ω_m : hold the baryon density fixed at $\omega_b = \Omega_b h^2 \approx 0.0224$, neglect (or fix) the neutrino density ω_ν , and derive the cold dark matter density as $\omega_c = \Omega_m h^2 - \omega_b - \omega_\nu$ for each sampled (H_0, Ω_m) . The primordial amplitude A_s enters directly as a `CAMB` input.

Noise model. Add a k -dependent, cosmic-variance-style diagonal covariance: $\hat{P}(k_i) = P(k_i) + \sigma_i \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, 1)$ and $\sigma_i = P(k_i) \sqrt{2/N_i}$, where the mode count $N_i = k_i^2 \Delta k V / (2\pi^2)$ for a chosen survey volume V (set $V = 10^9 (\text{Mpc}/h)^3$). A simpler fractional model $\hat{P}(k_i) = P(k_i)(1 + \varepsilon_i)$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, is acceptable as a baseline.

Preprocessing. Work in $\log k$ and feed standardized $\log P(k)$ to the summary network; raw $P(k)$ spans many orders of magnitude and is numerically poor as a direct input.

Budget. Precompute $\sim 10^4$ – 10^5 simulations offline as the simulator is computationally expensive.

References:

1. <https://people.ast.cam.ac.uk/~pettini/IntroCosmology/Lecture14.pdf>
2. <https://camb.readthedocs.io/en/latest/>
3. <https://arxiv.org/abs/1807.06209>
4. <https://tinyurl.com/2ektjkdu>

6. Gravitational Wave Source Parameter Estimation

Lecturer: Aayush (aayush.mishra@tu-dortmund.de)

When two black holes spiral together and merge, they emit gravitational waves (GWs) whose strain $h(t)$ encodes the physical properties of the binary. The frequency and amplitude evolution of the waveform depends on:

- m_1, m_2 : component masses (set the frequency evolution and merger time)
- χ_1, χ_2 : dimensionless *aligned* spin components along the orbital angular momentum (affect the waveform phasing)
- D : luminosity distance (sets the overall amplitude)
- ι : inclination of the orbital plane to the line of sight (sets the relative amplitude of the two polarizations)

The goal of this project is to use **BayesFlow** to infer the posterior over $\theta = \{m_1, m_2, \chi_1, \chi_2, D, \iota\}$ from a single noisy strain time series, using an amortized neural posterior estimator (NPE) with jointly trained summary and posterior networks.

Waveform. Generate the two polarizations $h_+(t), h_\times(t)$ with **pyCBC** using the aligned-spin, non-precessing approximant **IMRPhenomD** [2]. This model takes only the aligned spin components $\chi_1, \chi_2 \in [-1, 1]$; precession is out of scope.

Detector and extrinsic parameters. Assume a single detector and fix all extrinsic parameters. The measured strain is $h(t) = F_+ h_+(t) + F_\times h_\times(t)$, where the antenna-pattern factors $F_+, F_\times$ are computed once for a fixed sky position (α, δ) and polarization angle ψ (e.g. an optimally oriented source) and held constant across all simulations. Coalescence time t_c and phase φ_c are likewise fixed. Only the six parameters in θ are inferred.

Noise. Add *colored* Gaussian noise drawn from a realistic power spectral density, *not* white noise. Use **pyCBC**'s **aLIGOZeroDetHighPower** PSD via **pycbc.psd**, generate the noise with **pycbc.noise**, and form the observation as signal + noise on a segment of a few seconds at a sampling rate of 2048 or 4096 Hz, with a low-frequency cutoff $f_{\text{lower}} \approx 20$ Hz.

Preprocessing. *Whiten* the strain with the known PSD before passing it to the summary network — the raw colored-noise series buries the signal and is numerically poor as a direct input. Optionally work in the frequency domain and/or restrict to a short window around merger to reduce the input dimensionality.

Priors and conventions. Use independent priors as a starting point: $m_1, m_2 \in [10, 80] M_\odot$ with the ordering convention $m_1 \geq m_2$ enforced in both the prior and the simulator (without it the likelihood is symmetric under swapping the two bodies and the posterior becomes spuriously bimodal); $\chi_1, \chi_2 \in [-0.99, 0.99]$; $D \in [100, 1000] \text{ Mpc}$; and $\cos \iota \sim \mathcal{U}[-1, 1]$ (uniform in $\cos \iota$, not in ι).

Signal-to-noise control. Choose the distance prior so that typical signals have optimal SNR in the range ~ 10 – 30 . If D is allowed too large, most signals fall below the detection threshold, carry little information, and the posterior simply returns the prior.

Budget. Precompute $\sim 10^4$ – 10^5 simulations offline as the simulator is computationally expensive.

References:

1. <https://pycbc.org/pycbc/latest/html/>
2. <https://arxiv.org/abs/1508.07253>
3. <https://arxiv.org/abs/1809.02293>
4. <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.127.241103>

7. Inferring Initial Conditions in the 3-body Problem

Lecturer: Aayush (aayush.mishra@tu-dortmund.de)

The *3-body problem* is the task of predicting the motion of three mutually interacting particles under Newtonian gravity. Each body exerts a gravitational force on every other body, and their positions and velocities evolve according to Newton’s laws. The system is *non-integrable* and generically *chaotic*: nearby initial conditions diverge exponentially, so the forward map from initial state to later state is extremely sensitive to its input.

The goal of this project is to use **BayesFlow** to infer the posterior over the *initial conditions* of the system at $t = 0$ — the positions and velocities of the three bodies — given a noisy observation of the system’s evolution, using an amortized neural posterior estimator (NPE) with jointly trained summary and posterior networks. The central point is that for a chaotic system a *wide, possibly multimodal, but well-calibrated* posterior is the correct scientific result, not a failure of the method.

Setup and dimension. Work in the **2D planar** problem to begin with; the interesting chaotic and periodic regimes already live there. Optionally extend to 3D once the 2D pipeline works. Nondimensionalize: set $G = 1$, fix a mass unit (e.g. total mass $M = m_1 + m_2 + m_3 = 1$) and a length unit (e.g. a characteristic initial separation = 1). Standardize all network inputs.

Reference frame. Work in the **center-of-mass (COM) frame**: place the COM at the origin and set the total momentum to zero.

Distinct masses. Use three *distinct, fixed, labeled* masses (e.g. m_1, m_2, m_3 in fixed ratio such as 1:2:3, normalized so $\sum_i m_i = 1$). Keep body labels consistent between the simulator and the observation.

Simulator. Integrate the equations of motion with an adaptive, high-accuracy solver: `scipy.integrate.solve_ivp` with method `DOP853` and tight tolerances (e.g. `rtol=1e-10`, `atol=1e-12`). Add a small gravitational softening ε to the force law to tame close approaches, and restrict the prior to a regime that stays bounded over $[0, T]$ (reject initial conditions that eject a body or trigger a collision). Use energy conservation over the integration as a built-in check that the simulator is trustworthy.

Observation model. *Observe the trajectory*, not just the endpoint: record the state at a fixed set of intermediate times $t_1 < \dots < t_K \leq T$.

Timescale. Estimate the Lyapunov timescale empirically by first evolving two nearby initial conditions and measuring the exponential divergence rate λ . Ensure several observation times fall within the first Lyapunov time or so, where the information actually is, i.e., set $t \sim$ a few $/\lambda$.

Priors. Sample initial positions and velocities (in the COM frame, nondimensional units).

Noise. Add independent Gaussian observational noise with *separate* scales for position and velocity, since they carry different units: $\hat{x} = x + \sigma_x \varepsilon_x$, $\hat{v} = v + \sigma_v \varepsilon_v$, with σ_x, σ_v specified in the nondimensional units above and $\varepsilon \sim \mathcal{N}(0, I)$. If the trajectory is observed, apply the noise i.i.d. per recorded timestep.

References:

1. Valtonen, M., Karttunen, H. (2006) *The Three-Body Problem* (Cambridge University Press).
2. <https://webhomes.maths.ed.ac.uk/~douglas/cortina-v2.pdf>
3. <https://blbadger.github.io/3-body-problem.html>

8. Inferring Strong Gravitational Lens Parameters from Images

Lecturer: Aayush (aayush.mishra@tu-dortmund.de)

When a massive foreground galaxy lies almost directly between us and a more distant source galaxy, its gravity bends the source's light into arcs, multiple images, or a full *Einstein ring*. The geometry and brightness of this pattern encode the mass distribution of the lens. *Strong* lensing — the regime that produces multiple images rather than mild distortion — is one of the few direct probes of the total (including dark) matter in galaxies. The goal of this project is to use **BayesFlow** to infer the posterior over the lens and source parameters $\theta = \{\theta_E, e_1, e_2, \gamma_1, \gamma_2, x_s, y_s, R_s, \kappa\}$ given a single noisy image of a galaxy–galaxy strong lens, using simulation-based inference.

Parameters. θ_E is the Einstein radius (lens mass scale, sets the arc size); e_1, e_2 are the lens ellipticity components (shape and orientation); γ_1, γ_2 are external shear (line-of-sight distortion); x_s, y_s are the source position (sets how many images and where); R_s is the source size (arc thickness); and κ is the external convergence. For a simpler version, you may choose to fix $\kappa = 0$, what differences do you observe between the two version (with and without κ)?

Simulator. Generate images with **lenstronomy**. Use a Singular Isothermal Ellipsoid (SIE) for the main deflector, plus external **SHEAR** and a uniform **CONVERGENCE** sheet κ , lensing an elliptical Sersic source (**SERSIC_ELLIPSE**). Convolve with a fixed Gaussian PSF on a fixed pixel grid (e.g. 64×64 pixels at $0.05''/\text{pixel}$). Build the **ImageModel** once and reuse it across draws, varying only the parameter dictionaries. Fix the lens centroid (assumed known from the lens light), the source brightness, Sersic index, and source ellipticity; infer everything in θ .

Noise and SNR. Add a Gaussian background plus Poisson noise via **lenstronomy**'s utilities, using a fixed exposure time and background RMS. Tune the source amplitude so that typical images have peak SNR $\sim 10\text{--}30$.

Budget. Precompute $\sim 10^4\text{--}10^5$ simulations offline as the simulator is computationally expensive. Using a GPU will give a large speed-up during training. You may use Google colab or Kaggle for training with free GPUs.

References:

1. <https://lenstronomy.readthedocs.io/>
2. <https://arxiv.org/abs/1803.09746>
3. <https://arxiv.org/abs/1003.5567>

Models You need to use **BayesFlow** for estimating parameters of the statistical model of interest. Try to keep to a minimal working model so that you do not become distracted by too many moving parts in your project. If you need to make some simplifications to the scientific model even if they are against the scientific literature, do so. The goal is to implement a **BayesFlow** model and train it on data generated on simulators inspired by relevant scientific models, but not necessarily about implementing the state of the art scientific models.

Presentation

One presentation per group. The grade will usually be the same for all group members. However, in cases where the quality of presentation was very clearly different between group members, we may choose to give different grades for individual group members.

Submit your slides using the google form (<https://forms.gle/sw5immNjiQdvxQrV7>) before the deadline (**July 19, 23:59**). The deadline is strict and any submission after the deadline will not be considered. In case you face difficulty using the google form, you can submit via email to Aayush (aayush.mishra@tu-dortmund.de). The presentations should be in the PDF format named as *<group-number>.pdf*.

Presentation details:

- 10 minutes (group of 1–2) or 12 minutes (group of 3)
- The times are sharp. You will be interrupted after the scheduled time is over.
- First slide should contain title and group members' names.
- Introduce the topic.
- Explain the statistical model, describe how you fit it with BayesFlow.
- Discuss your results.
- Use captions and labels for all figures.
- Don't overload your slides. Think about what you consider good slides if you belong to the audience.
- Last slide should contain a TL;DR summary (or take-home-message) and contact info.
- No THANK YOU slide.
- After the presentation, teaching staff will ask questions.

IMPORTANT: At the end of the presentation we will ask you some questions. We may even ask questions about the general content of the SBI course not related to your particular project. These questions will be easy to answer if you followed the contents of the lectures and did the exercises during the semester. By answering these questions well you have an opportunity to improve your final grade. On the other hand, not knowing how to answer may worsen your grades.

Report

One report per group, one grade for the entire group.

Submit your report using the google form (<https://forms.gle/zPUXu3Uu6JQMza6Z6>) before the deadline (**August 23, 23:59**). The deadline is strict and any submission after the deadline will not be considered. The report should be in the zip format named as *<group-number>.zip* and contain both the report itself (as PDF) and the code with which to reproduce your results. In case you face difficulties with the google form, you can also submit via email to Aayush (aayush.mishra@tu-dortmund.de).

- **Content:**
 - Introduction: motivation, problem, modeling idea, illustrative figure
 - Data (if applicable): data collection, source, previous usage and analyses
 - Statistical model (simulator)
 - * Explicit proper priors

- * Describe how the data is generated
 - Approximator
 - * Describe the network architectures
 - * Explicit details (how many layers, activation functions, regularization, etc.)
 - Training
 - * Training budget (number of epochs, number of batches, batch size)
 - * Training regime (offline/online)
 - * Optimizer details, learning rate schedule, etc.
 - Diagnostics
 - * Convergence
 - * Simulation-based calibration
 - * Other relevant checks (posterior contraction, etc.)
 - Inference
 - * Fit the model to real data
 - * Show and discuss results
 - * Posterior predictive checks (if applicable)
 - Limitations and potential improvements
 - Conclusion
 - Reflection on own learnings
- Length: Maximal 10 pages of text (excluding title page, table of contents, figures, tables, and references). Your software code shall be submitted as a repository or zip-archive, so you generally don't have to print software code in the appendix.
 - Formatting: 12pt, 2.5cm margins, continuous text. We provide you a L^AT_EX template, but you are free to use any word processing software of your choice.

Grading

- The final grade will be determined by the teaching staff
- Presentation: The grade will usually be the same for all group members. However, in cases where the quality of presentation was very clearly different between group members, we may choose to give individual grades.
- Report: One grade for the entire group.
- Weight: 50% presentation, 50% report. Answering general SBI questions after presentations can have an impact as well.
- After presentation week: We will write you an email whether your group has passed (≥ 4.0) or failed (5.0) the presentation part. **If you fail the presentation, you will fail the entire course and then you do not need to submit the report.**
- If you submit your presentation slides but then don't show up for the presentation, we will consider this as a failed attempt. If you had registered but then didn't submit your slides, we will consider you to have dropped out from the course without adding a failed attempt in the Campusportal.